IE University

School of Science & Technology

Hierarchical N-Body Simulation of Galactic Dynamics in WebGPU

Enabling Scalable and Interactive Physics Simulations on Modern Web Platforms

Zaid Alsaheb

Bachelor of Computer Science & Artificial Intelligence

Supervised by: Professor Raul Perez Pelaez

1) Literature Review

Accurate simulation of galactic dynamics requires resolving gravitational interactions across large numbers of particles while maintaining numerical stability over long integration times. Achieving this balance has historically required substantial computational resources and specialized software, placing high-fidelity N-body simulations largely outside the reach of lightweight or widely accessible platforms. As a result, the relevant literature spans several traditionally separate domains, including astrophysical N-body methods, numerical integration techniques, GPU-accelerated computing, and browser-based visualization technologies.

This literature review is organized to clarify how these areas intersect and to highlight the technical gap addressed by this work. It first examines the physical and algorithmic foundations of gravitational N-body simulation, with particular attention to the computational scaling challenges that motivate hierarchical approximation methods such as Barnes–Hut. It then surveys prior efforts to accelerate N-body simulations using GPU architectures, with an emphasis on new emergent technologies that allow to achieve native performance in web browsers, drastically enhancing the range of devices that can perform these simulations.

Computational Constraints in Astrophysical Simulation

Over the past decades, progress in computational astrophysics has closely followed the performance improvements predicted by Moore's Law, with central processing unit (CPU) speeds increasing at a near-exponential rate. Once an algorithm was implemented, substantial performance gains could often be achieved simply by running existing code on newer hardware, with minimal additional development effort. However, as single-core CPU performance has plateaued (Sutter, 2005), this implicit scaling model has begun to break down. As a result, continued advances in computational astrophysics increasingly depend on exploiting parallelism and adapting algorithms to emerging computing architectures (Fluke et al., 2011).

One of the most significant architectural shifts has been the widespread adoption of graphics processing units (GPUs), which offer massive parallelism and high memory bandwidth. While GPUs provide the potential for orders-of-magnitude performance improvements (Owens et al., 2007), realizing these gains typically requires substantial algorithmic reformulation. Algorithms that scale well on serial or modestly parallel CPUs may perform poorly on highly parallel architectures if they involve irregular control flow or memory access pattern (Burtscher & Pingali, 2011).

Gravitational N-Body Simulations and Quadratic Scaling

Among the most computationally demanding problems in astrophysics are gravitational N-body simulations, which form the foundation of many studies in galactic dynamics and structure

formation (Binney & Tremaine, 2008). In such simulations, a galaxy is modeled as a system of particles (representing stars or dark matter) whose evolution is governed by their mutual gravitational interactions (Zwart et al., 2007).

According to Newton's law of universal gravitation, the force exerted on a particle of mass m_i by another particle of mass m_j is given by

$$\vec{F}_{ij} = G\frac{m_i m_j(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \tag{1}$$

Where $\vec{F}_{ij}$ denotes the gravitational force exerted on particle i by particle j, and G is the gravitational constant. The quantities m_i and m_j represent the masses of particles i and j, respectively. The vectors $\vec{r}_i$ and $\vec{r}_j$ give the positions of particles i and j in three-dimensional space.

The vector difference $\vec{r}_j - \vec{r}_i$ points from particle i toward particle j, while $|\vec{r}_j - \vec{r}_i|$ denotes the Euclidean distance between the two particles. The cubic power of the distance in the denominator ensures that the magnitude of the force follows an inverse-square law while preserving the correct force direction.

The total gravitational force acting on particle i is obtained by summing the pairwise force contributions from all other particles in the system, excluding self-interaction,

$$\vec{F}_i = \sum_{j \neq i}^{N} G\frac{m_i m_j(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \tag{2}$$

Evaluating this expression requires computing $N-1$ pairwise interactions per particle. Consequently, a direct implementation of gravitational force evaluation scales as $O(N^2)$ per simulation timestep. This quadratic scaling rapidly becomes computationally prohibitive as the number of particles increases, particularly when long integration times are required to observe large-scale dynamical phenomena such as spiral structure or halo evolution (Zwart et al., 2007).

Hierarchical Approximation and the Barnes–Hut Algorithm

To address the computational limitations imposed by direct force evaluation, a variety of approximation techniques have been developed to reduce the cost of N-body simulations while preserving essential physical behavior. One of the most influential of these methods is the Barnes–Hut algorithm, which exploits the hierarchical spatial structure of particle distributions to approximate gravitational interactions (Barnes & Hut, 1986).

Originally introduced by Barnes and Hut in 1986, the algorithm organizes particles into a tree structure, typically a quadtree in two dimensions or an octree in three dimensions. During force evaluation, distant groups of particles are approximated as a single effective mass located

at their center of mass, allowing subsets of the force summation to be replaced by a single interaction. An opening-angle parameter controls the accuracy of this approximation, providing a tunable tradeoff between computational efficiency and force accuracy.

By replacing many distant pairwise interactions with aggregate approximations, Barnes–Hut reduces the computational complexity of force evaluation from $O(N^2)$ to approximately $O(N \log N)$. This reduction makes the algorithm particularly well-suited for simulating large, collisionless systems such as galaxies, where global dynamical behavior is often of greater interest than exact short-range interactions.

Numerical Integration and Stability Considerations

In addition to force evaluation, the accuracy and long-term stability of N-body simulations depend critically on the numerical integration of the equations of motion. In gravitational N-body problems, the primary quantity of interest is the time evolution of the particle positions, which is governed by Newton's equations of motion. For each particle i, the total gravitational force resulting from interactions with all other particles determines its acceleration according to

$$\sum_j \vec{F}_{ij} = m_i \vec{a}_i \tag{3}$$

Where $\vec{a}_i$ is the acceleration of the particle i. The acceleration is related to the particle velocity and position through time derivatives,

$$\vec{a}_i = \frac{d\vec{v}_i}{dt}, \qquad \vec{v}_i = \frac{d\vec{r}_i}{dt} \tag{4}$$

Because these equations generally cannot be solved analytically for systems containing many interacting particles, their evolution must be approximated numerically. Numerical integration methods advance the particle positions and velocities forward in time using discrete timesteps of size Δt, approximating the continuous dynamics of the system.

One of the simplest numerical integration schemes is the forward Euler method (Kreyszig, 2011). In this approach, particle positions are updated using the current velocity according to

$$\vec{r}_i^{\,n+1} = \vec{r}_i^{\,n} + \vec{v}_i^{\,n} \Delta t \tag{5}$$

Where the superscript n denotes the discrete timestep, with $\vec{r}_i^{\,0}$ representing the initial particle position. Velocities are updated similarly using the acceleration computed from the forces at the current timestep. While the Euler method is computationally inexpensive and straightforward to implement, it suffers from poor numerical stability and does not conserve energy, leading to significant accumulated errors and unphysical behavior over long integration times.

As a result, astrophysical N-body simulations commonly employ symplectic integration schemes such as the Verlet or leapfrog methods, which are specifically designed to preserve the Hamiltonian structure of the equations of motion. These methods exhibit significantly improved long-term energy conservation and are therefore better suited for simulations of galactic dynamics that require stable evolution over many dynamical timescales (Springel, 2005).

In Barnes–Hut simulations, errors introduced by numerical integration interact with approximation errors arising from hierarchical force evaluation (Salmon & Warren, 1994). Prior studies have shown that, for many galactic-scale simulations, integration error can dominate force-approximation error when inappropriate timestepping schemes or integration methods are used (Springel, 2005). Consequently, careful selection of numerical integration techniques remains essential even when approximate force models are employed.

GPU Acceleration of Hierarchical N-Body Methods

The computational demands of gravitational N-body simulations have made them a natural target for graphics processing units (GPUs), which provide massive data parallelism and high memory bandwidth. Modern GPUs are designed with thousands of small, efficient cores capable of executing the same operation on many data elements simultaneously (Fluke et al., 2011). This architecture is particularly well-suited to the independent, particle-wise force computations inherent in N-body simulations (Nyland et al., 2009).

To leverage GPUs effectively, computational tasks are expressed in terms of small programs that run on the GPU cores called shaders. Originally, shaders were designed to compute visual effects for rendering pipelines, including vertex transformations, fragment coloring, and texture operations. Early GPU-based scientific computing therefore relied on repurposing graphics shaders for numerical tasks, encoding computation as rendering operations and storing data in textures (Owens et al., 2008).

Over time, GPU programming languages such as CUDA and OpenCL have generalized this model to allow general-purpose computation, enabling the parallel execution of tasks that are not directly related to graphics, including N-body simulations (Fluke et al., 2011; Nyland et al., 2009).

Despite these advances, hierarchical algorithms such as Barnes–Hut present persistent challenges for GPU architectures. Tree construction and traversal introduce irregular memory access patterns and branch divergence, both of which reduce parallel efficiency on SIMD-style hardware (Burtscher & Pingali, 2011; Karras, 2012). To mitigate these issues, prior work has proposed optimizations including linearized tree representations, stackless traversal methods, and the use of space-filling curves to improve memory coherence (Burtscher & Pingali, 2011;

Gaburov et al., 2010). While these techniques have enabled efficient native GPU implementations, they rely on low-level memory management and flexible data structures that have traditionally been inaccessible in browser-based computing environments.

Since the introduction of the Barnes–Hut algorithm, numerous modifications and alternative hierarchical methods have been proposed to improve accuracy and efficiency, including higher-order multipole expansions and the Fast Multipole Method (FMM) (LF & Rokhlin, 2001; Nyland et al., 2009; Wang, 2021). These approaches differ primarily in how distant particle interactions are approximated and aggregated. Despite these developments, Barnes–Hut remains widely used due to its favorable balance between computational cost, accuracy, and implementation complexity.

General Purpose GPU Computation in the Web

Modern web applications increasingly rely on GPU acceleration to deliver interactive, intelligent, and computation-heavy experiences directly in the browser. Access to GPU resources enables use cases such as real-time large-model reasoning, embedding generation, multimedia processing, and advanced data visualization without requiring users to install specialized software or manage local hardware (Congote et al., 2011; Sung et al., 2025; Usta, 2024). For users working across devices or within managed environments (e.g., students, researchers, enterprise teams), browser-based GPU access lowers friction, improves accessibility, and supports scalable, on-demand compute.

WebGL has long served as the primary mechanism for accessing GPU acceleration within web browsers. Standardized by the Khronos Group as a JavaScript binding to OpenGL ES, WebGL was designed to provide portable and efficient access to GPU hardware for real-time graphics rendering across a wide range of devices (Khronos Group, 2024a). Its success has enabled a broad ecosystem of browser-based visualization tools, interactive simulations, and educational applications.

Although WebGL exposes programmable shaders, its computational model is fundamentally centered around the graphics rendering pipeline. All computation must be framed in terms of vertex and fragment processing stages, with data represented as textures and intermediate results captured via framebuffers (Congote et al., 2011). As a consequence, general-purpose computation in WebGL requires reformulating numerical algorithms as sequences of rendering passes, often involving non-intuitive data layouts and multiple shader invocations to emulate iteration and state updates.

This graphics-centric abstraction has enabled a class of data-parallel algorithms to be executed efficiently in the browser, particularly those that map naturally onto image-based

representations. However, it imposes significant constraints on algorithms that require irregular memory access, dynamic data structures, or complex control flow (Congote et al., 2011) . In the context of gravitational N-body simulations, these constraints complicate the implementation of adaptive time stepping, hierarchical spatial decomposition, and recursive tree traversal, all of which are central to physically accurate Barnes–Hut simulations (Burtscher & Pingali, 2011).

As a result, prior WebGL-based N-body implementations have typically prioritized real-time performance and visual plausibility over numerical rigor. Common approaches include limiting particle counts, employing softened or approximate force models, reducing integration accuracy, or relying on fixed spatial grids rather than fully adaptive hierarchical trees (Sengupta et al., 2025). While such methods are effective for interactive visualization and outreach, they are not well suited to large-scale galactic simulations where long-term energy conservation and accurate force evaluation are essential.

WebGPU is a modern web standard designed to provide low-level, high-performance access to GPU hardware from within the browser. Unlike WebGL, which is built around a traditional graphics pipeline, WebGPU exposes explicit support for general-purpose GPU computation through compute shaders, storage buffers, and programmable pipelines (GPU for the Web Working Group, 2026). Its design closely mirrors contemporary native graphics and compute APIs such as Vulkan (Khronos Group, 2024b), Metal (Apple Inc., 2024), and Direct3D 12, enabling fine-grained control over memory layout, synchronisation, and parallel execution (Usta, 2024).

Central to WebGPU's computational model is the compute shader: a general-purpose GPU program that executes independently of the rendering pipeline. Compute shaders allow developers to express parallel workloads directly in terms of data processing rather than image generation, making them well suited to scientific computing tasks such as particle simulation, spatial partitioning, and force evaluation (Sung et al., 2025; Usta, 2024). This model enables algorithms to be structured in a manner similar to native GPU implementations, without the need for graphics-specific workarounds (Sengupta et al., 2025).

The explicit memory model and flexible buffer abstractions provided by WebGPU make it possible to represent complex data structures, including hierarchical trees and linearized spatial indices, directly on the GPU (Sung et al., 2025; Usta, 2024). These features are particularly relevant for Barnes–Hut simulations, which rely on dynamic tree construction and traversal to achieve sub-quadratic scaling. While such patterns were previously impractical in web environments, WebGPU enables their implementation with performance characteristics comparable to native GPU code, subject to browser and hardware constraints (Sengupta et al., 2025).

However, WebGPU's security-focused design imposes per-operation validation and command buffer submission overhead that compounds across many small dispatches. Maczan (Maczan, 2026) systematically characterised this overhead for LLM inference across four GPU vendors, three backends, and three browsers, finding a true per-dispatch cost of 24–36 µs on Vulkan and 32–71 µs on Metal, with substantial variation across implementations (2.2 × between Safari and wgpu-native on the same Metal backend). That work established that per-operation overhead, rather than kernel compute efficiency, is the dominant bottleneck for dispatch-heavy GPU workloads at small batch sizes. The present work extends this characterisation from machine learning inference to scientific simulation, where the dispatch pattern (a fixed pipeline of tree construction and force evaluation passes per timestep) differs fundamentally from the many small operations of a neural network forward pass.

2) Methodology

2.1 Research Design and Objectives

This work adopts a computational-methods design in which a gravitational N-body solver is implemented and evaluated with a focus on reproducibility, long-term numerical stability, and computational scalability beyond the $O(N^2)$ cost of direct summation. The solver is implemented in C++20 using the WebGPU C API and built from a single codebase targeting both native desktop execution (via backends such as wgpu-native and Dawn) and browser execution (compiled via Emscripten) (Khronos Group, 2024a). This dual-target approach enables interactive visualization as well as headless batch runs, allowing performance and numerical behavior to be evaluated under comparable conditions across platforms.

The physical and numerical foundations of the implementation draw directly from the literature on hierarchical N-body simulation and GPU parallelism. The gravitational force model adopts Newtonian gravity with Plummer-type softening to avoid the $\frac{1}{r^2}$ singularity and reduce spurious two-body relaxation in collisionless regimes (Binney & Tremaine, 2008). Accelerations are computed using a Barnes–Hut-style hierarchical approximation that reduces force evaluation from $O(N^2)$ to approximately $O(N \log N)$ (Barnes & Hut, 1986), with the primary code path constructing and traversing a Linear Bounding Volume Hierarchy (LBVH) entirely on the GPU using the parallel method of Karras (Karras, 2012). Time integration employs a second-order symplectic leapfrog scheme (kick–drift–kick), selected for its improved long-term energy behavior relative to forward Euler in gravitational systems (Springel, 2005). The compute platform, WebGPU, provides general-purpose parallel compute shaders that enable both native and browser deployment through the same low-level API surface.

The evaluation is structured around three research questions:

1. **Scalability**: How does runtime per timestep scale with N for a WebGPU Barnes–Hut implementation, and where are the bottlenecks within the GPU pipeline?
2. **Abstraction overhead**: What performance cost does the WebGPU abstraction layer impose relative to native Metal, and how does this vary across WebGPU implementations (wgpu-native, Dawn, browser)?
3. **Browser feasibility**: Can the same codebase, compiled to WebAssembly and running in a browser, achieve acceptable throughput and numerical consistency compared to native execution?

The choice of WebGPU as the compute platform is central to all three research questions; the following section provides a detailed justification.

2.2 WebGPU Platform Justification

The Barnes–Hut algorithm at $O(N \log N)$ requires a full tree rebuild and traversal at every timestep, each involving operations over the entire particle set. At the particle counts targeted by this work (N up to 10^5), these operations are dominated by data-parallel phases: computing bounding boxes, sorting Morton codes, constructing tree topology, and evaluating forces across all particles simultaneously. This degree of parallelism makes GPU execution essential for interactive frame rates, a conclusion broadly supported by the literature on GPU-accelerated N-body simulation (Fluke et al., 2011; Owens et al., 2007; 2008).

The specific choice of WebGPU over established GPU APIs such as CUDA and OpenCL is motivated by three factors. First, WebGPU is a cross-platform compute and rendering API that exposes general-purpose compute shaders and storage buffers through a unified interface, running natively on Vulkan (Khronos Group, 2024b), Metal (Apple Inc., 2024), and Direct3D 12 backends while also being deployable in web browsers (GPU for the Web Working Group, 2026; Khronos Group, 2024a). This portability is central to the research objective of evaluating N-body simulation across diverse execution environments, from native desktop applications to browser-based deployments. Second, WebGPU's compute shader model provides the primitives required for tree-based algorithms: random-access storage buffer reads and writes, atomic operations for bottom-up aggregation, and flexible workgroup dispatch (Sung et al., 2025; Usta, 2024). Third, WebGPU is an emerging W3C standard with active implementation across major browsers and native runtimes, positioning it as the successor to WebGL for GPU-accelerated web applications (Sengupta et al., 2025).

These advantages are accompanied by constraints that inform the implementation design. GPU-side computation in WebGPU is limited to 32-bit floating-point arithmetic, imposing a precision ceiling on force evaluation and integration. Buffer size limits and memory allocation patterns vary across devices and backends. Scheduling overhead for compute dispatches can be significant relative to kernel execution time at small N, and device variability across integrated and discrete GPUs affects both performance and available features (Sengupta et al., 2025). These constraints are explicitly tested through the platform-feasibility research question.

A comparative assessment of available GPU APIs clarifies the positioning of WebGPU. CUDA provides mature tooling and high performance for tree-based N-body codes (Burtscher & Pingali, 2011; Gaburov et al., 2010), but is locked to NVIDIA hardware and has no browser deployment path. OpenCL offers cross-vendor support on desktop but lacks browser integration and has seen declining adoption in favour of vendor-specific APIs. WebGL provides broad browser reach but was designed for rendering, not general-purpose computation: it

lacks compute shaders, storage buffers, and atomic operations, limiting it to fragment-shader workarounds for GPGPU tasks (Congote et al., 2011). WebGPU is uniquely positioned as both browser-deployable and compute-capable, making it the only viable API for evaluating an N-body solver across native and web environments from a single codebase. Table 1 summarises this comparison.

Criterion	**CUDA**	**OpenCL**	**WebGL**	**WebGPU**
Compute shaders	Yes	Yes	No	Yes
Storage buffers	Yes	Yes	No	Yes
Atomic operations	Yes	Yes	No	Yes
Browser deployment	No	No	Yes	Yes
Cross-vendor GPU	No	Yes	Yes	Yes
Native deployment	Yes	Yes	No	Yes
64-bit GPU float	Yes	Varies	No	No

Table 1: Comparison of GPU compute APIs against criteria required for hierarchical N-body simulation. WebGPU is the only API that combines compute shader support with browser deployability.

2.3 Physical Model and State Representation

This section describes the gravitational force model, the GPU data layout for particle state, and the precision strategy that governs the split between GPU computation and CPU diagnostics.

Each particle represents a mass element evolving under self-gravity in an isolated (open) domain. The simulation adopts a natural unit system in which the gravitational constant $G = 1$, the unit of mass M_0 is defined by the total system mass, and the unit of length r_0 is set by the characteristic scale of the initial conditions (e.g., the Plummer scale length a in Scenario B). Time is measured in units of $t_0 = \sqrt{r_0^3/(GM_0)}$, the free-fall timescale of the system. All quantities reported in subsequent sections (energies, momenta, and timescales) are expressed in these natural units unless stated otherwise. The softened acceleration of particle i due to all other particles is

$$\boldsymbol{a}_i = G \sum_{j \neq i} m_j \frac{\boldsymbol{r}_j - \boldsymbol{r}_i}{\left(\|\boldsymbol{r}_j - \boldsymbol{r}_i\|^2 + \varepsilon^2\right)^{\frac{3}{2}}} \tag{6}$$

where $\boldsymbol{r}_i$ and $\boldsymbol{r}_j$ are the position vectors of particles i and j, m_j is the mass of particle j, and ε is a softening length (default 0.5, configurable). The softening parameter introduces a Plummer-type potential that suppresses the $1/r^2$ singularity at short range, preventing divergent accelerations during close encounters (Binney & Tremaine, 2008). This formulation is equiv-

alent to treating each particle as a smooth mass distribution of characteristic radius ε rather than a point mass.

On the GPU, particle state is stored as packed four-component vectors (`vec4<f32>`), with each particle's mass occupying the w component of its position vector to yield a layout of (x, y, z, m). This packing reduces the number of buffer reads required during force evaluation compared to maintaining separate position and mass buffers. A Structure-of-Arrays (SoA) layout separating positions and masses was considered but rejected after analysis: the BVH force shader (which accounts for 95–99% of step time) never reads particle masses directly, instead using the aggregated node masses computed during the bottom-up pass. The shaders that do read particle mass (leaf initialisation, drift) always access position and mass together, so separating them would increase cache misses. The shaders that read only position (bounding box, Morton code) account for less than 1% of step time, making the potential bandwidth saving negligible. Table 2 summarises the storage layout for all three state arrays.

Component	`.x`	`.y`	`.z`	`.w`
`positions[i]`	x_i	y_i	z_i	m_i
`velocities[i]`	$v_{x,i}$	$v_{y,i}$	$v_{z,i}$	0
`accelerations[i]`	$a_{x,i}$	$a_{y,i}$	$a_{z,i}$	0

Table 2: Particle state layout using `vec4<f32>` packing. Mass is stored in the w component of the position vector to reduce buffer count and memory bandwidth during force evaluation.

All GPU kernels operate in 32-bit floating point, which maximises throughput and matches the capabilities of current WebGPU implementations. Diagnostic quantities (total energy and linear momentum) are computed on the CPU in double precision (64-bit) to reduce accumulation error over long integrations. This split reflects the practical precision-versus-performance trade-off inherent in WebGPU, where 64-bit GPU arithmetic is not available (Sengupta et al., 2025).

2.4 Time Integration

Time integration uses a fixed-timestep, second-order symplectic leapfrog integrator in kick–drift–kick (KDK) form (Verlet, 1967). In the following equations, the superscript n denotes the discrete timestep index, and $\boldsymbol{r}_i^n$, $\boldsymbol{v}_i^n$, $\boldsymbol{a}_i^n$ are the position, velocity, and acceleration vectors of particle i at step n respectively. With timestep Δt (default 10^{-4}), the update proceeds in three stages. First, a half-kick advances velocities by half a step using the current accelerations:

$$\boldsymbol{v}_i^{n+1/2} = \boldsymbol{v}_i^n + \frac{\Delta t}{2}\boldsymbol{a}_i^n \tag{7}$$

Next, a drift advances positions using the half-stepped velocities:

$$\boldsymbol{r}_i^{n+1} = \boldsymbol{r}_i^n + \Delta t \boldsymbol{v}_i^{n+1/2} \tag{8}$$

Accelerations $\boldsymbol{a}_i^{n+1}$ are then recomputed from the updated positions using the hierarchical force evaluation described in the following section. Finally, a second half-kick completes the velocity update:

$$\boldsymbol{v}_i^{n+1} = \boldsymbol{v}_i^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{a}_i^{n+1} \tag{9}$$

The leapfrog scheme is chosen for its well-known long-term stability in gravitational dynamics (Springel, 2005). Unlike forward Euler, which introduces secular energy drift proportional to Δt, the leapfrog is time-reversible and symplectic: it preserves the phase-space volume of the Hamiltonian system, producing bounded energy oscillations rather than monotonic growth (Binney & Tremaine, 2008). These properties make it substantially more suitable for long-horizon integrations.

The most expensive step in each integration cycle is the evaluation of accelerations $\boldsymbol{a}_i$, which is addressed through hierarchical approximation in the following section.

2.5 Hierarchical Force Evaluation

Rather than computing all $N(N-1)/2$ pairwise interactions directly, the solver groups distant particles into tree nodes and approximates each group by a single equivalent mass. This section describes the monopole approximation used to represent these groups and the opening criterion that determines when a node is sufficiently distant to be treated as a single body.

Each internal node of the tree stores a total mass $M = \sum_{i\in \text{ node}} m_i$ and a center of mass $\boldsymbol{R} = \left(\sum_{i\in \text{ node}} m_i \boldsymbol{r}_i\right)/M$, computed by aggregating over all particles contained in that node's subtree. When a node is accepted as a monopole, its gravitational contribution to particle i is

$$\boldsymbol{a}_{i,\text{node}} = GM\frac{\boldsymbol{R}-\boldsymbol{r}_i}{(\|\boldsymbol{R}-\boldsymbol{r}_i\|^2+\varepsilon^2)^{\frac{3}{2}}} \tag{10}$$

The tree is implemented as a binary Bounding Volume Hierarchy (BVH) constructed and traversed entirely on the GPU.

An internal node is accepted as a monopole when it is sufficiently small relative to its distance from the target particle. The opening criterion tests whether the squared maximum extent of the node's axis-aligned bounding box (AABB) satisfies $\text{maxExtent}^2/d^2 < \theta^2$, where $\text{maxExtent} = \max(\Delta x, \Delta y, \Delta z)$ and $d^2 = \|\boldsymbol{r}_i - \boldsymbol{R}\|^2$. This extent-based criterion reflects the non-cubic node shapes that arise in a BVH more accurately than a uniform half-width, and is expressed in squared form to avoid a per-node square root. Salmon and Warren provide rigorous error bounds for these opening criteria, showing that the conventional Barnes–Hut criterion can admit unbounded errors for $\theta \geq 1/3$ in pathological configurations (Salmon & Warren, 1994).

Figure 1 illustrates the geometry of both criteria. The default opening angle $\theta = 0.75$ is used as a practical balance between accuracy and performance. For comparison, GADGET-2 uses θ values in the range 0.5 to 0.7 for cosmological simulations where higher force accuracy is required (Springel, 2005), while the original Barnes and Hut paper used $\theta = 1.0$ (Barnes & Hut, 1986). The parameter sweeps in the evaluation protocol (see Section 2.11) systematically characterize the accuracy–performance trade-off across $\theta \in \{0.3, 0.5, 0.7, 1.0\}$.

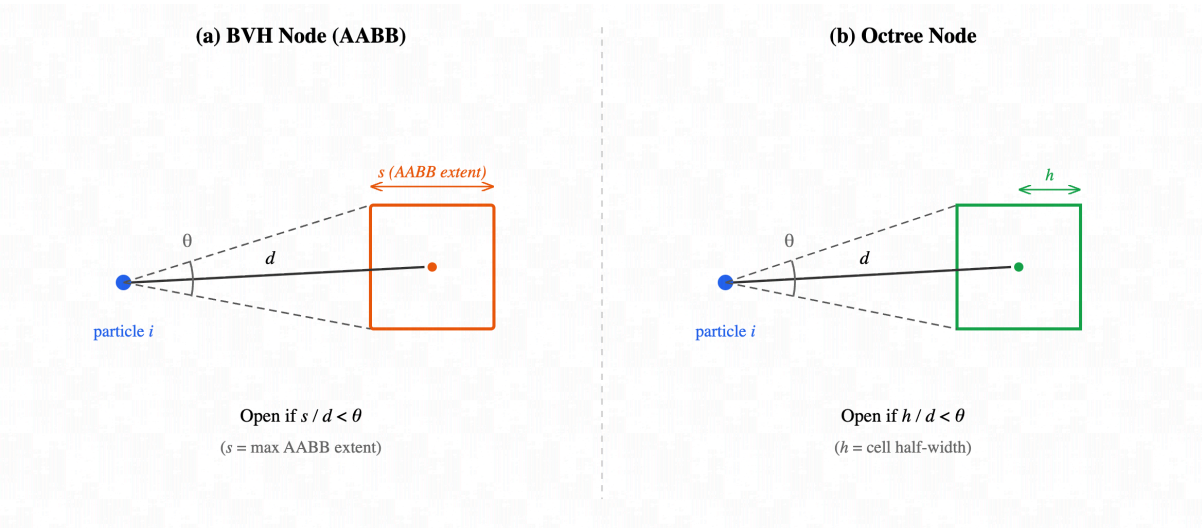


Figure 1: Geometry of the opening criterion. A node is approximated as a monopole when its angular size, as measured by extent/d, falls below the threshold θ. Left: BVH variant using maximum AABB extent. Right: octree variant using cell half-width.

2.6 Software Architecture and Execution Modes

The implementation is written in C++20 for host-side orchestration and physics, with WGSL for GPU compute and rendering shaders, using the WebGPU C API directly without wrapper libraries. The build system uses CMake with pinned dependency versions to ensure deterministic builds. Three build backends are supported: wgpu-native (gfx-rs contributors, 2024) (the Rust-based WebGPU implementation), Dawn (Google, 2024) (Google's WebGPU implementation), and Emscripten (which cross-compiles the C++ codebase to WebAssembly for browser deployment).

All physics operations (integration, tree construction, and force evaluation) are performed entirely on the GPU each step. The CPU is used only for diagnostic computation: at configurable intervals, positions and velocities are read back through staging buffers and energy and momentum are computed in double precision (64-bit). No per-step CPU physics overhead is incurred in the primary execution path.

The GPU-primary execution mode relies on a set of WebGPU compute shaders organized into a per-timestep pipeline, described in the following section.

2.7 WebGPU Compute Methodology

This section describes the GPU-side data layout, the compute shader organisation, and the per-timestep execution pipeline that together form the core of the solver.

Simulation state is stored in WebGPU storage buffers using the packed `vec4<f32>` layout described in Table 2: positions (with mass in the w component), velocities, and accelerations. The BVH is stored as a flat array of $2N - 1$ nodes, each containing a centre of mass, total mass, and axis-aligned bounding box. A uniform parameters buffer shares simulation parameters between host and shader code. In-place updates are used throughout: there is no double buffering of positions, velocities, or accelerations. Correct ordering between compute passes relies on WebGPU's implicit storage-buffer synchronisation between dispatches within a single command buffer submission. Bind groups are cached and reused across frames to avoid per-frame recreation overhead.

The solver comprises WGSL compute shaders in two groups: integration and force evaluation (direct-summation baseline and BVH-traversal), and LBVH construction in seven passes (Karras, 2012; Morton, 1966). Workgroup sizes are tuned per kernel (128 for force evaluation, 256 for integration and tree building; the workgroup size selection is discussed in the optimisation section below).

2.7.1 Per-timestep execution sequence

Each timestep is recorded into a single command encoder and submitted as one command buffer. The six sequential passes, illustrated in Figure 2, are: (1) a half-kick that advances velocities by $\Delta t/2$ using the current accelerations; (2) a drift that advances positions by Δt using the half-stepped velocities; (3) a full LBVH rebuild comprising seven sub-passes (global AABB reduction, Morton code generation, radix sort, Karras topology construction, leaf initialisation, and bottom-up aggregation); (4) BVH force evaluation, in which each particle traverses the tree iteratively using a fixed-depth explicit stack (depth 64, sufficient for all tested N); (5) a second half-kick that completes the velocity update; and (6) an optional diagnostics readback at configurable intervals, in which positions and velocities are copied to the CPU via staging buffers for double-precision energy and momentum computation.

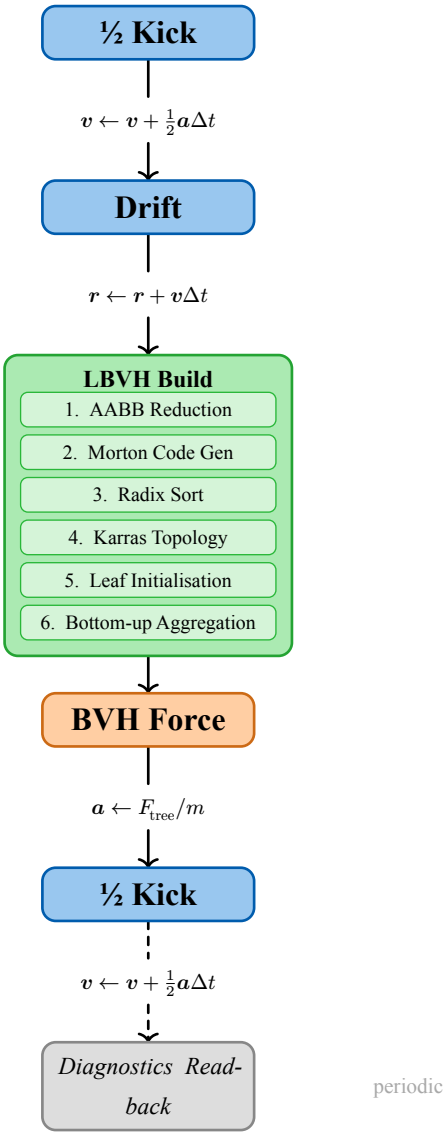


Figure 2: Per-timestep GPU execution pipeline for the leapfrog integrator. All passes are recorded into a single command buffer. The LBVH build comprises six sub-passes with implicit barrier synchronisation between each. Diagnostics readback occurs at configurable intervals.

2.7.2 GPU tree traversal

Because GPU compute shaders do not support recursion, tree traversal is implemented iteratively in the BVH force shader. One GPU thread is assigned per particle, and the thread maintains an explicit stack of node indices (maximum depth 64), beginning from the root. At each step, a node is either accepted as a monopole (if it is a leaf, or if the opening criterion is satisfied) and its gravitational contribution is accumulated, or it is expanded by pushing its two children onto the stack. This explicit-stack approach preserves the Barnes–Hut approximation structure while accommodating the lack of a call stack on GPU hardware and limiting branch divergence across threads within a workgroup (Burtscher & Pingali, 2011; Karras, 2012; Nyland et al., 2009).

2.7.3 GPU LBVH construction

The Linear Bounding Volume Hierarchy is built fully on the GPU each timestep, following the parallel construction method of Karras (Karras, 2012). The construction proceeds through seven compute dispatches, illustrated in Figure 4:

1. **Global bounding box reduction.** A two-pass parallel reduction over all particle positions computes the axis-aligned bounding box of the entire system. This bounding box defines the coordinate range used to normalise positions in the next step.
2. **Morton code generation.** Each particle's position is normalised to a $[0, 1023]^3$ integer grid within the global bounding box, and the three 10-bit integer coordinates are interleaved to produce a single 30-bit Morton code (Morton, 1966). The Morton code maps a three-dimensional position to a one-dimensional index along a Z-order (Morton) space-filling curve, so that particles that are spatially close in 3D tend to receive numerically similar codes. Figure 3 illustrates this principle in two dimensions: the Z-curve (red dashed line) visits grid cells in an order that preserves spatial locality, so that after sorting by Morton code, particles in the same cell are stored contiguously in the sorted array. The same principle extends to three dimensions with octant interleaving.

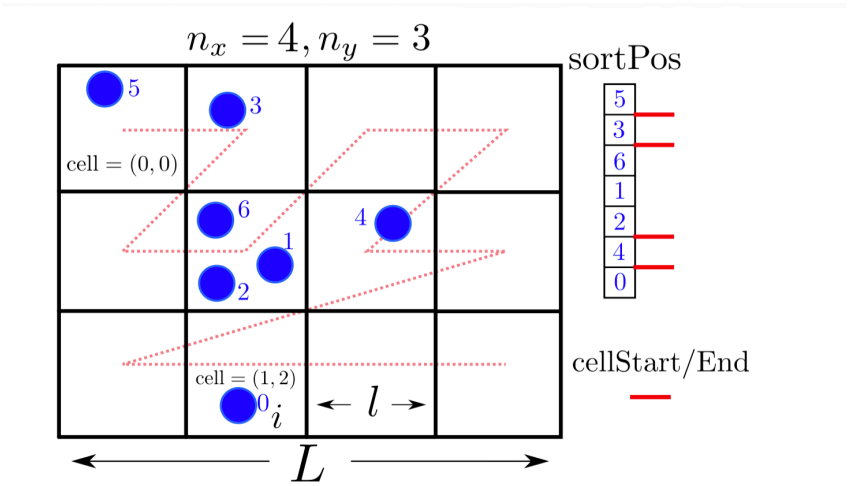


Figure 3: Spatial binning via a Z-order (Morton) space-filling curve in 2D. The dotted red line traces the curve through the grid; after sorting by Morton code, spatially adjacent particles are stored contiguously in the sorted array (right). Adapted from Peláez (Peláez, 2022).

1. **Radix sort.** The Morton codes and their associated particle indices are sorted into ascending order using a parallel radix sort. The radix sort processes the 30-bit keys in fixed-width digit passes (4 bits per pass), performing a prefix-sum histogram within each pass to determine output positions. This approach was chosen over the bitonic sort network (Batcher, 1968) used in an earlier version of the implementation, because radix sort achieves $O(N)$ work complexity for fixed-width keys and scales more predictably on GPU hardware. An

alternative is the onesweep radix sort, which is the most performant variant on native GPU APIs, but its reliance on fine-grained device-scope atomic operations makes it difficult to implement efficiently in WebGPU, where atomics are limited to workgroup and storage-buffer scope.

2. **Karras topology construction.** The sorted Morton codes define a binary radix tree whose internal structure is determined entirely by the codes themselves. For each internal node i, the Karras algorithm computes a direction and range by examining the *delta function* $\delta(i, j)$, defined as the number of leading zero bits in the bitwise XOR of Morton codes k_i and k_j. Two codes that share a long common prefix (high δ) correspond to particles that are spatially close, because their Morton codes agree on the most significant bits of their interleaved coordinates. The algorithm determines each internal node's children by finding the split position within its range where the common prefix length changes. Duplicate Morton codes (which arise when two particles fall in the same grid cell) are handled by appending the particle index as a tie-breaker to ensure a unique ordering (Karras, 2012).
3. **Leaf initialisation.** Each leaf node is assigned the position, mass, and a point AABB corresponding to its sorted particle.
4. **Bottom-up aggregation.** Internal-node bounding boxes and centres of mass are computed in a bottom-up pass. An atomic counter per internal node tracks how many of its two children have been processed; the second thread to arrive at a node computes the merged AABB and mass-weighted centre of mass from both children, ensuring correctness without explicit synchronisation barriers.

The resulting BVH is immediately traversable without any CPU-side construction or data upload, eliminating what would otherwise be a per-step CPU–GPU transfer bottleneck.

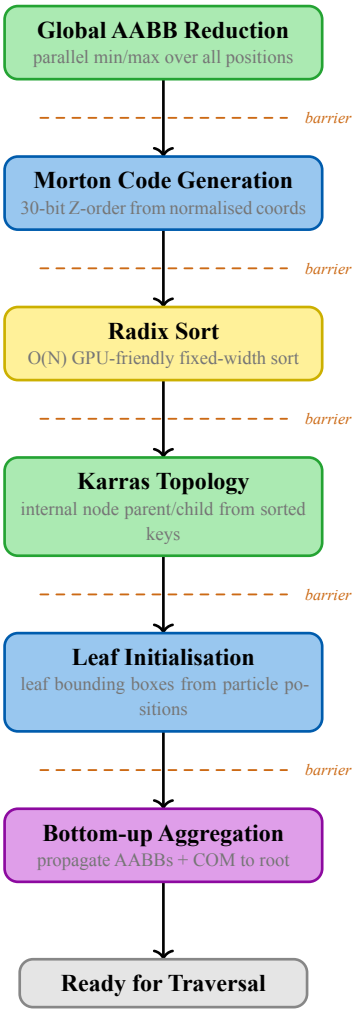


Figure 4: LBVH construction pipeline. Six compute dispatches build the tree entirely on-device. Dashed lines represent implicit storage-buffer barriers between passes. The atomic-counter aggregation in pass 6 ensures correct bottom-up propagation of bounding boxes and centres of mass.

2.8 Force Traversal Optimisation

Profiling revealed that BVH force evaluation accounts for 95–99% of total step time across all tested N, with LBVH construction contributing less than 1% even at $N = 100000$. Optimisation efforts were therefore focused exclusively on the traversal shader. Four optimisations were applied, each benchmarked independently before combining.

The baseline traversal shader performs three operations per node visit: a full 64-byte struct read from global memory, an opening-criterion computation involving approximately 20 floating-point operations including transcendental functions, and a stack-based depth-first traversal with no spatial ordering. Each represents a different bottleneck class (memory bandwidth, ALU throughput, and cache efficiency respectively), and the optimisations target all three.

Precomputed opening radius. The opening criterion depends only on node-intrinsic properties (centre of mass, AABB bounds, total mass) that are invariant across particle interactions. In the baseline, these are recomputed for every particle–node pair. The optimised version precomputes a scalar opening radius per node during the bottom-up aggregation pass and stores it in an otherwise unused field. This eliminates redundant computation across all particle interactions, yielding a 1.74× speedup at $N = 100000$.

Compact traversal nodes. The force shader reads a 64-byte BVH node per visit but uses only 28 bytes (centre of mass, opening radius, child indices). A compaction pass copies these fields into a 32-byte traversal buffer, reducing memory bandwidth per node visit. This provides an additional 3% improvement at large N.

Workgroup size tuning. Testing workgroup sizes of 64, 128, and 256 with otherwise identical code showed that 128 threads per workgroup (four SIMD groups on the Apple M2) provided the best balance between register pressure and occupancy for N up to 50,000. This parameter is hardware-dependent and would require re-tuning on other GPUs.

Morton-ordered particle access. In the baseline, GPU thread i processes particle i, so adjacent threads may handle spatially distant particles that traverse different parts of the tree, causing poor cache utilisation. The optimised version uses the Morton-code sort order already computed during LBVH construction: thread i processes the particle at sorted index i, so adjacent threads handle spatially nearby particles that visit similar tree paths. This improves L1/L2 cache hit rates during traversal, yielding an additional 20% speedup at $N = 100000$.

Two additional approaches were investigated and rejected. Near-far child ordering (pushing the farther child first so the nearer subtree is processed first) caused cache thrashing from the extra node reads, making force computation 10–30% slower. Warp-coherent traversal using subgroup operations was not feasible because WGSL subgroup support is limited to experimental extensions on the Dawn backend.

N	Baseline (ms)	Optimised (ms)	Speedup
1,000	5.8	6.9	0.84×
5,000	9.9	8.0	1.24×
10,000	11.3	11.5	∼1.0×
50,000	109.5	67.2	1.63×
100,000	396.2	183.9	2.15×

Table 3: Combined effect of all four traversal optimisations. The overhead of the compaction pass and sort-index lookup causes a small regression at $N = 1000$; the crossover where optimisations break even is approximately $N = 3000$.

2.9 Rendering and Interactive Operation

In interactive mode, particles are rendered as instanced billboard quads with additive blending. The vertex shader reads positions directly from the physics storage buffers, avoiding per-frame data upload. An ImGui overlay provides interactive control of simulation parameters and displays real-time diagnostics. In headless mode, rendering is skipped entirely for pure throughput measurement.

Having described the solver and its implementation, the following section specifies the benchmark scenarios used to evaluate it.

2.10 Initial Conditions and Benchmark Scenarios

No external astronomical datasets are used. All experiments are generated from synthetic initial conditions and produce derived outputs (trajectories, diagnostic scalars, and timing logs). This enables controlled, repeatable comparisons across parameter sweeps. Each experiment is fully specified by its simulation parameters: scenario type, seed, N, Δt, θ, softening ε, and step count. Three benchmark scenarios are defined, with initial particle distributions shown in Figure 5.

2.10.1 Scenario A: two-body circular orbit

Scenario A places two equal-mass particles ($m = 1000$ each, $N = 2$) separated by $d = 10$ units along the x-axis, with tangential velocities along the z-axis computed for a softened circular orbit:

$$v = \sqrt{\frac{Gmd^2}{2(d^2 + \varepsilon^2)^{\frac{3}{2}}}} \tag{11}$$

This configuration provides the simplest possible validation of integrator correctness: with the correct timestep and softening, the two particles should maintain a stable circular orbit indefinitely under the leapfrog scheme. Validation is quantitative rather than visual: the energy drift $\Delta E/\ |E(0)|$ over the full integration is measured directly, and departures from circularity can be isolated without confounding effects from hierarchical force approximation at large N.

2.10.2 Scenario B: Plummer sphere

A Plummer sphere is a spherically symmetric, self-gravitating stellar system whose density falls off smoothly with distance from the centre. It was first introduced by Plummer (Plummer, 1911) as an empirical fit to the light profiles of globular clusters, and its density profile is given by $\rho(r) \propto \left(1 + r^2/a^2\right)^{-5/2}$, where a is a scale length that sets the size of the core. Because the Plummer model has known analytic properties — including closed-form expressions for the potential, escape velocity, and distribution function — it is widely used as a standard test case for N-body codes (Aarseth et al., 1974; Binney & Tremaine, 2008). Deviations from the

expected equilibrium behaviour provide a direct diagnostic of force accuracy and integration stability.

Scenario B generates a Plummer sphere with $N \in [10^3, 10^5]$ (depending on hardware) and scale length $a = 5$. Particle radii are sampled via the inverse cumulative distribution function

$$r = \frac{a}{\sqrt{u^{-2/3} - 1}} \tag{12}$$

with u clamped to $[0.001, 0.999]$. Angular coordinates are isotropic: $\cos(\theta)$ is drawn uniformly and the azimuthal angle ϕ is drawn uniformly on $[0, 2\pi)$. Particle speeds are sampled via rejection sampling using

$$g(q) = q^2(1 - q^2)^{\frac{7}{2}} \tag{13}$$

against the local escape velocity

$$v_e = \sqrt{\frac{2GM}{\sqrt{r^2 + a^2}}} \tag{14}$$

with isotropic velocity directions. This scenario tests tree accuracy and long-term stability in a compact three-dimensional distribution. The Plummer softening parameter ε in the force model is conceptually related to the Plummer scale length a in the initial conditions, though the two are independently configurable. The key sweep variables are θ, ε, and Δt; the primary limitation is that the spherical geometry does not emphasize disk morphology such as spirals and bars.

2.10.3 Scenario C: rotating exponential disk (galaxy-like morphology test)

Scenario C generates a rotating disk with $N \in [10^4, 10^5]$ to evaluate long-term evolution and visually interpretable galactic dynamics. The surface density profile follows an exponential distribution (Freeman, 1970), with particle radii drawn from an exponential distribution (rate parameter 0.08) and clamped to a maximum of 50 units, and azimuthal angles drawn uniformly. Vertical height is drawn from a normal distribution $\mathcal{N}(0, 0.3)$ scaled by $\frac{1}{1+0.5r}$ to produce a thinner disk at larger radii. Particle masses are drawn uniformly from $[0.5, 2.0]$.

Circular velocities are assigned using an approximate enclosed-mass estimate. For particles with $r > 0.1$, the tangential velocity is

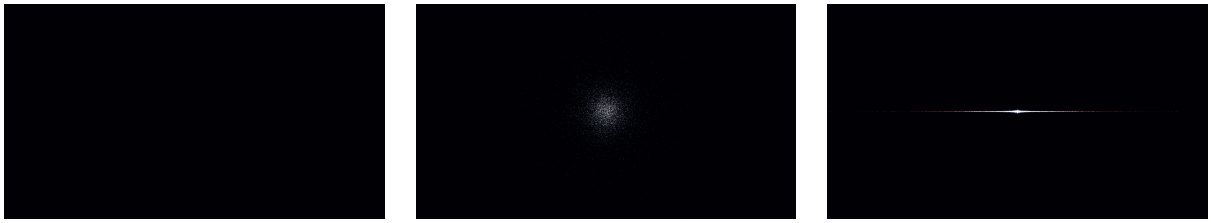
$$v = 0.5\sqrt{\frac{M_{\text{enclosed}}}{r}} \tag{15}$$

with the velocity directed tangentially. This simplified dynamical setup is not a full multi-component Milky Way model: the enclosed-mass estimate is approximate, and no bulge or halo component is included. Nevertheless, the disk geometry provides a morphologically rich test case in which the formation and persistence of spiral structure, bars, and other large-

scale features serve as qualitative validation of the solver's long-term behavior. The key sweep variables are disk scale length, thickness, velocity dispersion, θ, and Δt.

2.10.4 Sampling and robustness across seeds

Because initial conditions are stochastic, robustness is assessed by repeating runs with different random seeds and comparing diagnostics and timing. Runs are considered valid if they complete without NaN or overflow values and produce consistent parameter logs. Deliberately unstable configurations (such as excessively large Δt) are retained as documented failures for robustness reporting rather than silently excluded.



(a) Two-body orbit (b) Plummer sphere (c) Exponential disk

Figure 5: Initial particle distributions for the three benchmark scenarios. (a) Scenario A: two-body orbit. (b) Scenario B: Plummer sphere with $N = 10000$. (c) Scenario C: exponential disk with $N = 50000$.

2.11 Evaluation Protocol

The evaluation is designed to answer the three research questions by comparing the WebGPU solver against a native Metal baseline and across multiple WebGPU implementations, using a consistent benchmarking protocol throughout.

The primary baseline is UniSim (Gomes, 2024), an open-source Barnes–Hut N-body solver written directly against the Metal API. Because UniSim runs on the same Apple M2 GPU using the same Metal driver as the WebGPU implementations, the performance difference isolates the overhead introduced by the WebGPU abstraction layer. A direct $O(N^2)$ summation path within the WebGPU solver serves as a secondary baseline for characterising the crossover point at which hierarchical force evaluation becomes beneficial.

The primary metric is runtime per timestep (milliseconds per step), decomposed into three components: tree build time (GPU LBVH construction), force evaluation time (BVH traversal), and integration time (kick and drift dispatches). This decomposition identifies which phase of the pipeline dominates at each particle count. For the cross-backend comparison, the abstraction overhead is quantified as the ratio of WebGPU ms/step to native Metal ms/step at matched N and parameters. For the browser comparison, the browser wall-clock time minus the native GPU time yields the fixed per-step scheduling overhead.

Energy drift, defined as $\Delta E(t) = |E(t) - E(0)| \ / \ |E(0)|$, is reported as a secondary observation for $N \leq 5000$ (where potential energy is computed via direct pair summation). Momentum conservation is monitored throughout. These numerical quality metrics characterise the 32-bit precision floor of WebGPU rather than serving as a primary research question.

Accurate GPU timing requires the CPU to wait for GPU work to complete before reading the clock. The synchronisation mechanism differs across backends: wgpu-native provides a blocking device poll, while Dawn and Emscripten use a buffer-map fence (a small staging buffer whose map callback fires only after all prior GPU work completes). Two timing modes are used: whole-step timing, in which a single command encoder records the entire timestep and the GPU is synchronised once at the end (used for total ms/step comparisons), and per-phase timing, in which separate command encoders and GPU synchronisations are issued per phase (used for the LBVH pass breakdown, at the cost of additional synchronisation overhead).

All measurements follow the benchmarking protocol described in the experiments section: 50 warmup steps discarded, 100 measured steps, with mean $\pm$ standard deviation, 95% confidence interval, and coefficient of variation reported for each configuration (Maczan, 2026). All initial conditions are generated deterministically (default seed 42), and the execution environment is recorded alongside each run.

3) Experiments

This section specifies the hardware environment and the concrete experiment configurations used to evaluate the solver against the three research questions defined in the methodology. All experiments use the headless batch mode of the simulation, eliminating rendering overhead and producing deterministic CSV logs for post-processing.

3.1 Experimental Platform

All experiments were executed on a single workstation whose configuration is summarised in Table 4. The GPU adapter name and WebGPU backend are reported by the application at startup and recorded alongside each run.

Component	**Specification**
GPU / CPU	Apple M2 (8-core GPU, 4P+4E CPU, unified memory)
RAM	16 GB unified memory
Operating System	macOS 26.2 (Darwin 25.2.0)
Native WebGPU	wgpu-native (Rust-based, Metal backend) via WebGPU-distribution v0.2.0
Native WebGPU (alt.)	Dawn (Google, Metal backend) via WebGPU-distribution v0.2.0
Browser (Chromium)	Chrome 146.0.7680.178 (arm64)
Browser (WebKit)	Safari 26.2
Metal baseline	UniSim (Gomes, 2024) (native Metal Barnes–Hut)
Compiler	Apple Clang 17.0.0, C++20, Release (`-O3 -DNDEBUG`)
Timing	CPU wall-clock via `std::chrono::high_resolution_clock` (sub-µs resolution), bracketing backend-specific GPU fences: `wgpuQueueOnSubmittedWorkDone` + poll (wgpu-native), buffer-map fence (Dawn / Emscripten). `--sync-timing` for total ms/step; `--benchmark-passes` for per-component breakdown.

Table 4: Hardware and software configuration. All experiments run on the same Apple M2 system. The four WebGPU implementations and the native Metal baseline all use the same Metal graphics driver, isolating the overhead of each abstraction layer.

Each run is executed in headless batch mode, with all simulation parameters (scenario type, N, Δt, θ, softening ε, integrator, tree type, force method, step count, and random seed) specified at invocation. The CSV output records twelve columns per step: step number, simulation time, kinetic energy, potential energy (zero when $N > 5000$), total energy, energy drift, three momentum components (p_x, p_y, p_z), and three timing components (tree build, force evaluation, and integration in milliseconds).

3.2 Benchmarking Protocol

The timing methodology follows established practices for GPU benchmarking (Maczan, 2026). Each configuration is warmed up by discarding the first 50 steps, during which pipeline compilation, buffer allocation, and caching stabilise. Timing is then collected over the subsequent 100 steps and reported as mean $\pm$ standard deviation, with the 95% confidence interval computed via the t-distribution. The coefficient of variation ($\mathrm{CV} = \sigma/\mu$) is reported to quantify run-to-run stability; configurations with $\mathrm{CV} > 10\%$ are flagged and investigated. For the cross-backend comparison (Group 6), positions are held frozen (forces are computed but not applied) to isolate dispatch and scheduling overhead from physics-dependent variation in tree structure. This frozen-state protocol ensures that all backends execute identical GPU workloads, so that measured differences reflect only the overhead of the WebGPU implementation and host-side scheduling.

3.3 Experiment Groups

The experiments are organised into seven groups, each targeting one or more research questions or characterising numerical quality. Table 5 provides a summary.

Group	Scenario	Variable	Runs	RQ
1: Two-body validation	A (two-body)	$\Delta t \times$ integrator	10	Qual.
2a: N-scaling	B (Plummer)	$N \in \{100\text{–}10^5\}$	9	RQ1, RQ3
2b: Theta sweep	B (Plummer)	$\theta \in \{0.3, 0.5, 0.7, 1.0\}$	4	Qual.
2c: Timestep sweep	B (Plummer)	$\Delta t \in \{5 \times 10^{-5}\text{–}5 \times 10^{-3}\}$	5	Qual.
2e: Softening sweep	B (Plummer)	$\varepsilon \in \{0.1\text{–}2.0\}$	5	Qual.
3a: Disk N-scaling	C (disk)	$N \in \{10^4\text{–}10^5\}$	5	RQ1, RQ3
4: Direct vs tree crossover	B (Plummer)	force method $\times N$	12	RQ1
5: Native vs browser	B (Plummer)	execution platform $\times N$	9	RQ3
6: Cross-backend comparison	B (Plummer)	WebGPU impl $\times N$	12	RQ2, RQ3
7: LBVH pass breakdown	B (Plummer)	per-pass timing $\times N$	5	RQ1

Table 5: Summary of experiment groups, swept variables, and targets. Qual. denotes numerical quality characterisation, reported as a secondary observation rather than a primary research question.

3.3.1 Group 1: Two-Body Orbit Validation (Scenario A)

This group verifies integrator correctness using the two-body orbit configuration ($N = 2, m = 1000$ each). The timestep Δt is swept over $\{0.0001, 0.0005, 0.001, 0.005, 0.01\}$ for both the leapfrog and Euler integrators, yielding ten runs of 50,000 steps each. All runs use the GPU

BVH tree path with $\theta = 0.75$, $\varepsilon = 0.5$, and seed 42. The primary diagnostic is the energy drift $\frac{\Delta E(t)}{|E(0)|}$ over the full integration, with secondary attention to momentum magnitude and orbit stability.

3.3.2 Group 2: Plummer Sphere Parameter Sweeps (Scenario B)

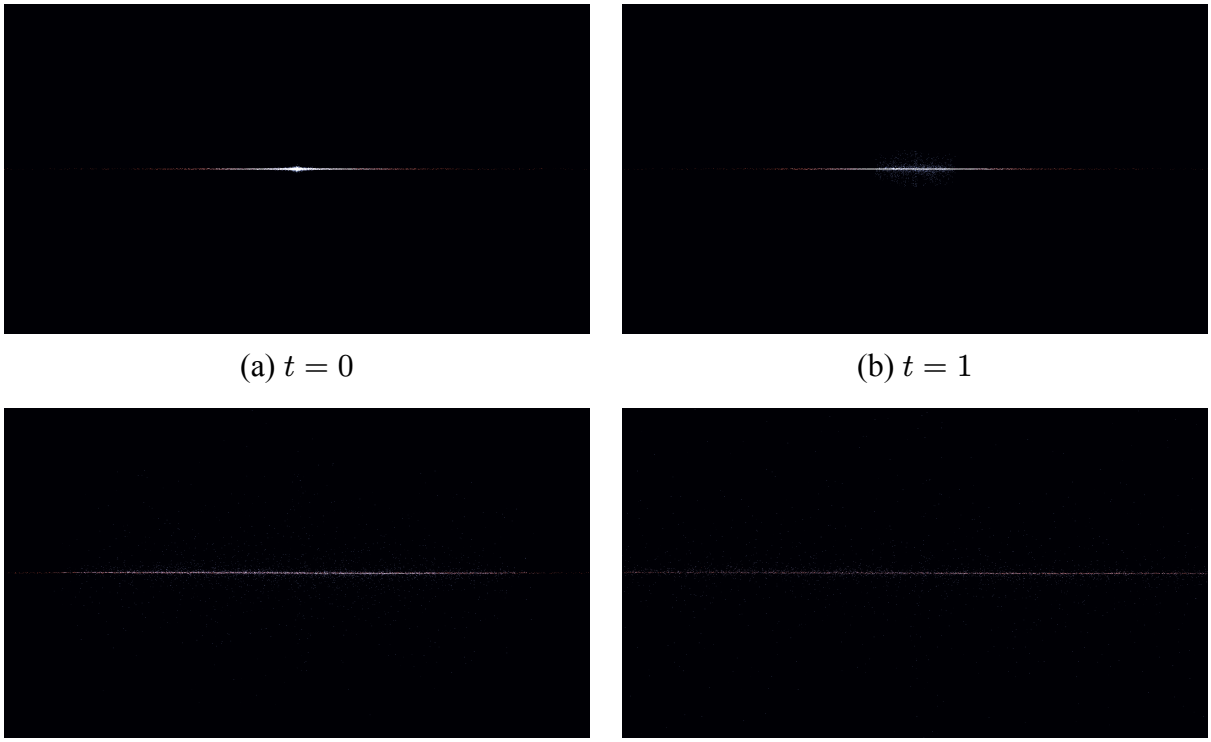
The Plummer sphere scenario provides the primary quantitative testbed because it has symmetric initial conditions and known analytic equilibrium properties. Four sub-groups isolate individual parameter effects.

Sub-group 2a (N-scaling) varies N over $\{100, 500, 1000, 2000, 5000, 10000, 25000, 50000, 100000\}$ with all other parameters fixed at defaults ($\Delta t = 0.001$, $\theta = 0.75$, $\varepsilon = 0.5$, leapfrog, GPU LBVH). Each run executes 1,000 steps. This sub-group directly addresses RQ1 (scalability) and RQ3 (feasibility at large N).

Sub-groups 2b, 2c, and 2e isolate the effects of individual parameters at fixed $N = 5000$ (chosen because potential energy is computed at this size, enabling full energy tracking). Sub-group 2b sweeps $\theta \in \{0.3, 0.5, 0.7, 1.0\}$; sub-group 2c sweeps $\Delta t \in \{5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}\}$; and sub-group 2e sweeps $\varepsilon \in \{0.1, 0.25, 0.5, 1.0, 2.0\}$. All parameter-sweep runs execute 5,000 steps.

3.3.3 Group 3: Rotating Disk Scaling (Scenario C)

The disk scenario targets large-N scalability and qualitative morphological assessment. Sub-group 3a scales N over $\{10000, 25000, 50000, 75000, 100000\}$ with default parameters and 1,000 steps each, measuring runtime scaling and timing decomposition. Morphological evolution (spiral arm formation, bar instability) is assessed through interactive-mode visualisation at selected timesteps (Figure 6) and reported descriptively.



(a) $t = 0$ (b) $t = 1$

(c) $t = 5$ (d) $t = 10$

Figure 6: Morphological evolution of the rotating exponential disk (Scenario C, $N = 50000$). Panels show the face-on particle distribution at four simulation times, illustrating the development of spiral structure.

3.3.4 Group 4: Direct vs Tree Crossover

To identify the particle count at which hierarchical force evaluation becomes faster than direct $O(N^2)$ summation, this group runs both the direct summation and tree-based force evaluation at $N \in \{100, 200, 500, 1000, 2000, 5000\}$ using the Plummer sphere scenario. Each configuration runs for 500 steps with default physics parameters.

3.3.5 Group 5: Native vs Browser Execution

To evaluate WebGPU's portability promise (RQ3), the same GPU LBVH Barnes–Hut configuration used in Group 2a is executed in a browser environment via Emscripten (Emscripten contributors, 2024). The native C++ codebase is cross-compiled to WebAssembly (WASM) using Emscripten with two key flags: `-sASYNCIFY`, which transforms synchronous C++ code into asynchronous form so that the simulation can yield control to the browser's event loop between timesteps (required because browsers do not allow long-running synchronous code on the main thread), and `-sALLOW_MEMORY_GROWTH=1`, which permits the WebAssembly linear memory to grow dynamically as particle count increases rather than requiring a fixed-size allocation at compile time. The resulting WebAssembly module runs in a headless Chromium instance on the same hardware, using the browser's WebGPU implementation (backed by the same Metal driver). Particle counts sweep $N \in \{100, 500, 1000, 2000, 5000, 10000, 25000, 50000, 100000\}$

with all other parameters matching Group 2a ($\Delta t = 0.001$, $\theta = 0.75$, $\varepsilon = 0.5$, leapfrog, GPU LBVH, 1000 steps).

Two timing metrics are collected: (1) wall-clock milliseconds per step, computed from the console-log timestamps emitted every 100 steps, which captures all overhead including event-loop scheduling and Emscripten asyncify yields; and (2) GPU-side timing from the sampled per-step tree/force/integrate breakdowns logged to the console. Energy drift at the final step is compared to the native run at each N to verify numerical consistency across platforms. A GPU command-buffer flush was required after each compute dispatch in the browser path to prevent command coalescing from stalling the pipeline; this fix had no effect on native execution.

3.3.6 Group 6: Cross-Backend Implementation Comparison

To characterise the performance impact of the WebGPU implementation layer itself, the same simulation configuration is executed across four WebGPU implementations on the same Apple M2 hardware, all backed by the Metal graphics API: wgpu-native (the default native backend), Dawn (Google, 2024) (Google's WebGPU implementation), Chrome (browser WebGPU via Emscripten), and Safari (browser WebGPU via Emscripten). This design isolates the overhead introduced by each implementation from the GPU compute itself, since all four paths execute identical WGSL shaders on the same Metal driver. Recent work on WebGPU dispatch overhead has shown that implementation choice within the same backend can produce up to 2.2× variation in per-dispatch cost (Maczan, 2026), making this comparison directly relevant to understanding the platform overhead reported in Group 5. Particle counts sweep $N \in \{1000, 10000, 100000\}$ using the frozen-state benchmarking protocol described above (50 warmup steps, 100 measured steps, positions held constant). For each implementation and N, the mean milliseconds per step, standard deviation, 95% confidence interval, and coefficient of variation are reported.

3.3.7 Group 7: LBVH Construction Pass Breakdown

To identify which phases of the LBVH construction pipeline dominate total tree-build time, per-pass timing is collected at $N \in \{1000, 5000, 10000, 50000, 100000\}$ using CPU-side timing markers placed between each compute dispatch. The seven LBVH sub-passes (global AABB reduction, Morton code generation, radix sort, Karras topology construction, leaf initialisation, bottom-up aggregation) and the force evaluation pass are timed individually. This decomposition reveals whether the sort phase, the topology construction, or the bottom-up aggregation is the primary bottleneck, and how the balance shifts with N.

4) Results and Analysis

This section presents results organised by the three research questions: RQ1 (scalability and pipeline bottlenecks), RQ2 (WebGPU abstraction overhead relative to native Metal), and RQ3 (browser feasibility). All timing values are mean milliseconds per step over 100 measured steps after 50 warmup steps, reported with standard deviation and coefficient of variation. Energy drift is reported as $\Delta E = |E(t) - E(0)| \ / \ |E(0)|$.

4.1 RQ1: Scalability and Pipeline Bottlenecks

4.1.1 Runtime Scaling with Particle Count

Table 6 presents the mean runtime per step and its three-component decomposition for the GPU LBVH Barnes–Hut path across the Plummer sphere scenario. Total runtime increases from 7.0 ms at $N = 1000$ to 182.8 ms at $N = 100000$.

N	ms/step	± std	CV	Tree (ms)	Force (ms)	Integrate (ms)
1,000	7.04	1.66	0.24	0.38	6.63	0.04
5,000	7.87	2.99	0.38	0.25	7.59	0.03
10,000	10.25	0.94	0.09	0.32	9.91	0.03
50,000	67.53	1.95	0.03	0.91	66.51	0.11
100,000	182.85	3.33	0.02	0.96	181.72	0.17

Table 6: Performance summary with timing decomposition (Plummer sphere, leapfrog, $\theta = 0.75$, $\varepsilon = 0.5$, wgpu-native/Metal). Force evaluation dominates at all N, accounting for 94–99% of total step time.

Force evaluation dominates at all particle counts, accounting for 94% of step time at $N = 1000$ and 99% at $N = 100000$. Tree construction (the seven-pass LBVH pipeline) remains below 1 ms even at $N = 100000$, indicating that the parallel construction method is efficient and that further optimisation efforts should target the BVH traversal shader rather than the tree-build pipeline.

4.1.2 LBVH Construction Breakdown

Table 7 decomposes tree construction into its seven individual passes. All six construction passes remain approximately constant with N (1.3–4.6 ms each), confirming that the LBVH pipeline scales well. The radix sort is the most expensive construction pass at 4.4–4.7 ms. Force evaluation, shown in the final column for reference, is the component that drives the overall scaling behaviour.

N	AABB	Morton	Sort	Karras	Leaf	Aggr.	Force
1,000	1.51	1.34	4.65	1.41	1.38	1.43	3.19
5,000	1.46	1.35	4.44	1.46	1.34	1.39	8.27
10,000	1.47	1.37	4.41	1.37	1.44	1.45	13.62
50,000	1.46	1.33	3.57	1.37	1.33	1.36	63.04
100,000	1.42	1.35	4.60	1.40	1.37	1.38	176.57

Table 7: Per-pass timing (ms) for the LBVH construction pipeline and force evaluation. Construction passes are approximately constant with N; force evaluation drives the overall scaling.

4.1.3 Direct vs Tree: Speed–Accuracy Trade-off

Table 8 compares runtime and energy drift between direct $O(N^2)$ summation and the tree-based $O(N \log N)$ path. Direct summation is faster at all tested particle counts, reflecting the high degree of GPU parallelism available for the regular, branch-free direct computation compared to the irregular memory access patterns of tree traversal. However, the tree path produces substantially lower energy drift: at $N = 100000$, direct summation yields $\Delta E = 2.01$ while the tree path achieves $\Delta E = 0.079$. The relevant crossover is therefore not one of runtime but of *accuracy*: the tree path sacrifices throughput for force accuracy that keeps the simulation physically meaningful over long integrations.

N	Direct (ms)	Tree (ms)	Direct drift	Tree drift
1,000	2.14	6.76	2.25×10^{-2}	6.45×10^{-5}
5,000	4.53	7.83	1.21×10^{-1}	2.40×10^{-4}
10,000	6.08	11.11	2.11×10^{-1}	9.26×10^{-3}
50,000	37.63	67.31	9.88×10^{-1}	6.14×10^{-2}
100,000	132.49	171.22	2.01	7.88×10^{-2}

Table 8: Direct vs tree force evaluation: runtime and energy drift (Plummer sphere, leapfrog, $\theta = 0.75$). Direct summation is faster but accumulates substantially more energy drift, particularly at large N.

4.2 RQ2: Abstraction Overhead

4.2.1 WebGPU vs Native Metal

Table 9 compares the WebGPU solver (wgpu-native/Metal) against the native Metal Barnes–Hut baseline (UniSim (Gomes, 2024)) on the same Apple M2 hardware. Both implementations use the same Metal graphics driver; the difference reflects the overhead of the WebGPU abstraction layer.

N	Metal (ms)	WebGPU (ms)	Ratio (WebGPU/Metal)
1,000	2.94	7.04	2.4×
5,000	10.09	7.87	0.78×
10,000	21.35	10.25	0.48×
50,000	121.77	67.53	0.55×
100,000	516.78	182.85	0.35×

Table 9: WebGPU (wgpu-native) vs native Metal (UniSim (Gomes, 2024), stabilised fork (Alsaheb, 2026)) Barnes–Hut performance on the same Apple M2. At $N \geq 5000$ the WebGPU implementation outperforms the Metal baseline, reflecting the efficiency of the fully GPU-resident LBVH pipeline.

At $N = 1000$, WebGPU is 2.4× slower than native Metal, reflecting per-dispatch overhead at small workloads. At $N \geq 5000$, the WebGPU implementation is consistently faster, reaching 2.8× faster at $N = 100000$ (182.85 ms vs 516.78 ms). This advantage reflects differences in tree construction and traversal strategy — the fully GPU-resident LBVH with the optimised traversal shader described in the methodology outperforms UniSim's approach at scale — rather than WebGPU being inherently faster than Metal. The key finding for RQ2 is that the WebGPU abstraction layer imposes no measurable overhead once GPU compute dominates at $N \geq 5000$.

4.2.2 Cross-Backend Implementation Comparison

Table 10 compares four WebGPU implementations on the same hardware, all backed by the Metal API. This isolates the overhead introduced by each implementation from the GPU compute itself (Maczan, 2026).

Implementation	N=1K (ms)	N=10K (ms)	N=100K (ms)	CV (100K)
Dawn	1.47	8.56	274.57	0.01
wgpu-native	7.04	10.25	182.85	0.02
Chrome	35.86	35.99	219.96	0.26
Safari	34.10	53.18	311.27	0.01

Table 10: Per-step runtime across four WebGPU implementations (Plummer sphere, $\theta = 0.75$, frozen-state protocol). All four use the Metal backend on Apple M2. Dawn is fastest at small N; wgpu-native scales best to large N.

The results reveal substantial variation across implementations. Dawn achieves the lowest per-dispatch overhead (1.47 ms at $N = 1000$) but scales to 274.57 ms at $N = 100000$, while wgpu-native starts higher (7.04 ms) but scales better (182.85 ms). The two browser implementations show a fixed floor of approximately 34–36 ms at small N, attributable to the Emscripten asyncify event-loop yield, above which they diverge: Chrome scales to 219.96 ms while Safari reaches 311.27 ms. These findings are consistent with Maczan's observation that

implementation choice within the same backend produces significant performance variation (Maczan, 2026).

4.3 RQ3: Browser Feasibility

Table 11 compares native wgpu-native execution against browser execution via Chrome for the same simulation configuration. At small N, the browser wall-clock time is dominated by a fixed overhead of approximately 29 ms from the Emscripten asyncify mechanism. As N grows and GPU compute time increases, this fixed cost becomes a diminishing fraction: at $N = 50000$ the browser matches native performance (67.77 ms vs 67.53 ms), and at $N = 100000$ the browser adds only a 1.2× overhead (219.96 ms vs 182.85 ms).

N	Native (ms)	Chrome (ms)	Overhead	Native drift
1,000	7.04	35.86	5.1×	6.38×10^{-5}
5,000	7.87	35.97	4.6×	2.85×10^{-4}
10,000	10.25	35.99	3.5×	9.31×10^{-3}
50,000	67.53	67.77	1.0×	6.06×10^{-2}
100,000	182.85	219.96	1.2×	7.71×10^{-2}

Table 11: Native (wgpu-native) vs browser (Chrome) wall-clock ms/step and energy drift. The browser overhead converges from 5.1× at $N = 1000$ to 1.2× at $N = 100000$.

4.4 Numerical Quality

Energy drift is reported as a secondary observation characterising the 32-bit precision floor of WebGPU rather than a primary research question. The opening angle sweep at $N = 5000$ (Table 12) shows identical drift ($\Delta E/\ |E(0)| \approx 1.84$) across all tested θ values, confirming that 32-bit floating-point precision, not the tree approximation, is the dominant error source. Softening in the range 0.1–2.0 has no measurable effect on runtime and produces only modest variation in drift (1.82–2.00). Momentum is conserved to within 0.1% over 5,000 steps.

θ	Mean ms/step	Final drift
0.3	3.46	1.84
0.5	3.65	1.84
0.7	3.66	1.84
1.0	3.49	1.84

Table 12: Effect of opening angle θ on runtime and energy drift (Plummer sphere, $N = 5000$). Drift is insensitive to θ, indicating 32-bit precision dominates.

4.5 Summary of Findings

Force evaluation accounts for 94–99% of step time, with the LBVH construction pipeline remaining below 1 ms even at $N = 100000$ (RQ1). The WebGPU abstraction overhead relative

to native Metal is 2.4× at $N = 1000$ but the WebGPU solver outperforms the Metal baseline at $N \geq 5000$, with substantial variation across WebGPU implementations (RQ2). Browser execution adds a fixed 29 ms overhead that becomes negligible at large N, with Chrome matching native throughput at $N = 50000$ (RQ3). The 32-bit precision floor is the binding constraint on numerical fidelity. These findings are interpreted in the following discussion section.

5) Discussion

5.1 Interpretation of Results

5.1.1 Scalability and Pipeline Structure

The scaling from 7.04 ms to 182.85 ms over a 100× increase in N is driven almost entirely by force evaluation, which accounts for 94–99% of step time. At small N the GPU is underutilised and dispatch overhead dominates; as N grows, the parallel hardware absorbs the increasing traversal workload. The LBVH construction pipeline, by contrast, remains below 1 ms at all tested particle counts, with individual passes approximately constant in N. The implication for practitioners is that optimisation efforts should target the BVH traversal shader, not the tree-build pipeline. The direct-versus-tree comparison reinforces this: direct summation is faster at all tested N (reflecting the high degree of GPU parallelism available for its regular, branch-free computation pattern), but the tree path produces 25× lower energy drift at $N = 100000$, making the hierarchical approach essential for physically meaningful long integrations.

5.1.2 Abstraction Overhead

The finding that the WebGPU solver outperforms the native Metal baseline (UniSim (Gomes, 2024)) at $N \geq 5000$ requires careful interpretation. The advantage does not imply that WebGPU is inherently faster than Metal; rather, the fully GPU-resident LBVH pipeline (radix sort, Karras topology construction, and bottom-up aggregation, all on-device) avoids the CPU–GPU coordination overhead present in UniSim's tree construction approach. The performance gap widens with N (0.48× at $N = 10000$, 0.35× at $N = 100000$), confirming that the benefit is structural rather than incidental. The cross-backend comparison reveals that implementation choice within the same Metal backend produces substantial performance variation: Dawn achieves the lowest per-dispatch overhead (1.47 ms at $N = 1000$) but scales to 274.57 ms at $N = 100000$, while wgpu-native starts higher but scales to 182.85 ms. This is consistent with Maczan's finding that WebGPU implementation choice can produce up to 2.2× variation in per-dispatch cost (Maczan, 2026). The practical implication is that researchers selecting a WebGPU backend for compute-heavy workloads should prefer wgpu-native for large-N throughput and Dawn for low-latency small dispatches.

5.1.3 Browser Platform Viability

The browser deployment path adds a fixed overhead of approximately 29 ms per step from the Emscripten asyncify event-loop yield, but this cost does not grow with N. At small particle counts (5.1× overhead at $N = 1000$), the asyncify cost dominates; as GPU compute time increases, the fixed cost becomes a diminishing fraction. At $N = 50000$ the browser matches native throughput (67.77 ms vs 67.53 ms), and at $N = 100000$ the overhead is only 1.2×.

The practical implication is that for scientifically meaningful particle counts ($N \geq 10000$), the browser platform approaches native performance, and at $N \geq 50000$ the overhead is negligible. This positions WebGPU as viable for interactive, accessible N-body simulation without specialised hardware, subject to the 32-bit precision constraint (Sengupta et al., 2025).

5.1.4 Numerical Quality

The theta-sweep results ($\Delta E/\ |E(0)| \approx 1.84$ at all tested θ values) confirm that 32-bit floating-point precision, not the tree approximation or opening angle, is the dominant constraint on numerical fidelity. This means that investing in higher-order multipole expansions or smaller opening angles will not improve energy conservation until 64-bit GPU arithmetic becomes available in WebGPU. For practical use, the leapfrog integrator remains the correct choice: it conserves momentum to within 0.1% over 5,000 steps and maintains stable orbits, consistent with its theoretical properties (Binney & Tremaine, 2008; Springel, 2005). The observed Plummer sphere expansion is physical rather than numerical, consistent with known behaviour of finite-N systems with approximate forces (Binney & Tremaine, 2008).

5.2 Comparison with Published Results

Direct comparison with CUDA-based Barnes–Hut implementations is limited by differences in hardware generation, floating-point precision (64-bit in CUDA studies versus 32-bit here), tree topology (octree versus BVH), and opening criteria. Nevertheless, several structural comparisons are informative.

Burtscher and Pingali report CUDA Barnes–Hut performance of approximately 10 billion body-body interactions per second on an NVIDIA GTX 280 (Burtscher & Pingali, 2011). At $N = 100000$ with $\theta = 0.75$, this corresponds to roughly 10 ms per step on 2010-era hardware. The WebGPU implementation requires 182.85 ms per step at the same N, reflecting the dominance of the BVH traversal shader (99% of step time) and the irregular memory access patterns of LBVH traversal compared to octree traversal. The comparison across hardware generations and tree topologies is not directly informative, but it indicates that the WebGPU compute shader model can support the complete solver pipeline at large N, with throughput determined primarily by traversal efficiency rather than platform overhead.

Gaburov, Bédorf, and Portegies Zwart report tree-construction overhead of approximately 15–25% of total step time in their CUDA octree implementation (Gaburov et al., 2010). The LBVH construction pipeline in this work accounts for less than 1% of step time (0.96 ms out of 182.85 ms at $N = 100000$), substantially lower than the CUDA reference. This reflects the efficiency of the fully GPU-resident construction method: by performing radix sort, topology

construction, and bottom-up aggregation entirely on-device, the LBVH approach eliminates the CPU–GPU transfer costs incurred by CUDA implementations that build the tree on the CPU.

Nyland, Harris, and Prins report direct $O(N^2)$ GPU throughput of approximately 10–30 GFLOP/s for N-body problems on contemporary NVIDIA hardware (Nyland et al., 2009). The WebGPU direct-summation path in this work achieves 4.53 ms per step at $N = 5000$ (equivalent to $\frac{N^2}{t} \approx 5.5 \times 10^9$ interactions per second), which is competitive given the integrated GPU architecture of the M2 and the 32-bit precision.

5.3 Limitations

The results should be interpreted in light of several limitations that bound their external validity.

The most significant constraint is that all experiments were conducted on a single Apple M2 system. Although the cross-backend comparison (Group 6) demonstrates that software-side variation across four WebGPU implementations is substantial, all four use the same Metal graphics driver on the same hardware. GPU performance characteristics vary across vendors; the Apple M2 is an integrated GPU with unified memory, which eliminates CPU–GPU transfer costs that would be present on discrete GPU systems, while its compute unit count is modest compared to dedicated NVIDIA or AMD GPUs. The timing results and scaling behaviour reported here may therefore not generalise to other hardware configurations or GPU backends (Vulkan, Direct3D 12).

A second fundamental constraint is the limitation of WebGPU to 32-bit floating-point arithmetic in compute shaders. As the results demonstrate, this single-precision ceiling is the dominant constraint on numerical fidelity, exceeding the effects of the opening angle and timestep size for the configurations tested. The effect is most pronounced at small N, where round-off error accumulates over many steps without the statistical averaging that occurs in large-N systems. Relatedly, the force evaluation uses only the monopole term of the multipole expansion; quadrupole and higher-order terms, which are standard in production codes such as GADGET-2 (Springel, 2005), would reduce the force approximation error at a given θ and potentially allow larger opening angles for the same accuracy target.

On the methodology side, all experiments use a global fixed timestep, whereas production codes commonly employ adaptive individual timestepping that assigns each particle a step proportional to its local dynamical timescale (Springel, 2005). The absence of adaptive timestepping means that the timestep must be chosen conservatively for the densest region of the simulation, which may be unnecessarily small for the majority of particles. Additionally, energy drift as a quantitative metric is available only for $N \leq 5000$, where direct pair summation

for potential energy is computationally feasible; for larger N, numerical quality is assessed indirectly through kinetic energy trends and momentum conservation.

6) Future Work

The limitations identified in the discussion suggest several concrete directions for extending this work, each tied to specific performance or capability goals.

6.1 Adaptive Individual Timestepping

The fixed global timestep used in this implementation requires a conservative choice of Δt that accommodates the densest region of the simulation. Adaptive individual timestepping, as implemented in GADGET-2 (Springel, 2005), assigns each particle a timestep proportional to a local dynamical timescale (such as $\sqrt{\frac{\varepsilon}{|a_i|}}$) and synchronises particles using block timestepping. Implementing this in WebGPU compute shaders would require per-particle step bookkeeping in storage buffers and more complex dispatch logic, but would significantly improve accuracy in the dense cores of Plummer spheres and disk centres without penalising the timestep for particles in diffuse outer regions.

6.2 Cross-Hardware and Cross-Browser Benchmarking

The cross-backend comparison in this work (Group 6) tested four WebGPU implementations (wgpu-native, Dawn, Chrome, and Safari) on the same Apple M2 hardware and found substantial cross-implementation variation, but all four use the same Metal graphics driver. Extending the benchmarks to non-Apple hardware — discrete NVIDIA and AMD GPUs using the Vulkan backend, and Intel Arc using Direct3D 12 — would determine whether the scaling characteristics and relative implementation rankings observed here generalise across GPU architectures. Firefox (once WebGPU is enabled) represents another untested browser implementation. Mobile devices (smartphones and tablets), where GPU scheduling, thermal throttling, and memory bandwidth differ from desktop systems, are a further priority: the approximately 29 ms per-step asyncify overhead observed in browser execution may vary on devices with different event-loop scheduling characteristics. Such testing would extend the findings of Sengupta et al. (Sengupta et al., 2025) to hierarchical N-body simulation across diverse consumer hardware.

6.3 Higher-Order Multipole Expansion

Extending the monopole approximation to include quadrupole moments in the BVH nodes would reduce the force approximation error at a given θ, potentially allowing the use of larger opening angles (and correspondingly faster traversal) for the same accuracy target. The additional per-node storage (a symmetric 3×3 tensor per node) fits within WebGPU storage buffer limits, and the extra arithmetic per interaction is modest. This extension would bring the force accuracy closer to production codes and enable more meaningful comparison with GADGET-2′s multipole expansion (Springel, 2005).

6.4 64-Bit Precision When Available

The results demonstrate that 32-bit GPU arithmetic is the dominant constraint on numerical quality. As the WebGPU specification evolves, optional 64-bit floating-point support in compute shaders (`f64`) may become available on supporting hardware. Benchmarking the precision-versus-performance trade-off with 64-bit force evaluation would determine whether the energy conservation improvement justifies the throughput reduction, and would enable direct integrator-quality comparisons (leapfrog vs Euler) that are currently confounded by the precision difference between GPU and CPU paths.

6.5 Multi-Component Galaxy Models

The rotating disk scenario (Scenario C) uses a simplified single-component model. Extending the initial conditions to include a central bulge, an exponential disk, and a dark matter halo, following standard multi-component galaxy models (Binney & Tremaine, 2008), would enable comparison with observational rotation curves and provide a more physically realistic testbed for the solver. The simulation architecture already supports heterogeneous particle masses and could accommodate multiple components with minimal code changes, primarily in the initial-condition generator.

7) Conclusion

This work presented a hierarchical N-body solver for galactic dynamics implemented entirely in WebGPU, featuring a fully GPU-resident Linear Bounding Volume Hierarchy (LBVH) constructed and traversed on-device each timestep. The implementation was evaluated across three benchmark scenarios (two-body orbit, Plummer sphere, and rotating exponential disk) with systematic parameter sweeps targeting three research questions: scalability and pipeline bottlenecks, WebGPU abstraction overhead relative to native Metal, and browser feasibility. Numerical quality under 32-bit floating-point precision was characterised as a cross-cutting secondary observation.

7.1 Scalability and Pipeline Bottlenecks (RQ1)

Total runtime increases from 7.04 ms per step at $N = 1000$ to 182.85 ms at $N = 100000$, with force evaluation accounting for 94–99% of step time across all particle counts. The six-pass LBVH construction pipeline remains below 1 ms even at $N = 100000$, with individual construction passes approximately constant in N, confirming that the parallel construction method scales efficiently and that the BVH traversal shader is the primary optimisation target. Direct $O(N^2)$ summation is faster than tree-based evaluation at all tested particle counts, but accumulates substantially more energy drift: at $N = 100000$, direct summation yields $\Delta E = 2.01$ compared to 0.079 for the tree path, demonstrating that the hierarchical approach trades throughput for the force accuracy required to keep simulations physically meaningful over long integrations.

7.2 Abstraction Overhead (RQ2)

Comparison against a native Metal Barnes–Hut baseline (UniSim (Gomes, 2024)) on the same Apple M2 hardware shows that the WebGPU abstraction layer imposes a 2.4× overhead at $N = 1000$, where per-dispatch cost dominates. At $N \geq 5000$ the WebGPU implementation is consistently faster, reaching 2.8× faster at $N = 100000$ (182.85 ms vs 516.78 ms), reflecting the efficiency of the fully GPU-resident LBVH pipeline rather than an inherent speed advantage of WebGPU over Metal. A four-way comparison across WebGPU implementations (wgpu-native, Dawn, Chrome, and Safari) reveals substantial cross-implementation variation consistent with Maczan (Maczan, 2026): Dawn achieves the lowest per-dispatch overhead (1.47 ms at $N = 1000$) while wgpu-native scales best to large N.

7.3 Browser Feasibility (RQ3)

Browser execution via Emscripten WebAssembly adds a fixed overhead of approximately 29 ms per step from asyncify event-loop scheduling. This cost dominates at small N (5.1×

overhead at $N = 1000$) but diminishes as GPU compute time grows: at $N = 50000$ the browser matches native throughput (67.77 ms vs 67.53 ms), and at $N = 100000$ the overhead is $1.2\times$ (219.96 ms vs 182.85 ms). The overhead is additive and constant, not multiplicative: the scaling behaviour observed in native execution is preserved in the browser, supporting WebGPU's viability as a platform for browser-based scientific simulation subject to the 32-bit precision constraint (Sengupta et al., 2025).

7.4 Contribution

The results demonstrate that WebGPU's compute shader model is sufficient for a complete scientific computing workload: parallel tree construction, hierarchical force evaluation, and symplectic integration, benchmarked across four WebGPU implementations and compared against a native Metal baseline. The WebGPU solver matches or exceeds native Metal performance at scientifically relevant particle counts, and browser execution converges to native throughput as GPU compute dominates. By building from a single C++/WGSL codebase that targets both native desktop and browser environments, this work shows that accessible, interactive galactic dynamics simulation is achievable without specialised hardware or vendor-locked APIs. The performance characteristics reported here, together with the identified 32-bit precision limitations, provide a reference point for future work on GPU-accelerated scientific computing in WebGPU as the specification and its implementations continue to mature.

References

Aarseth, S., Henon, M., & Wielen, R. (1974). A Comparison of Numerical Methods for the Study of Star Cluster Dynamics. *\aap*, *37*(1), 183–187.

Alsaheb, Z. (2026,). *UniSim fork: stabilised for large-N benchmarking*. https://github.com/haxybaxy/unisim

Apple Inc. (2024,). *Metal: Apple's low-level graphics and compute API*. https://developer.apple.com/metal/

Barnes, J., & Hut, P. (1986). A hierarchical O(N log N) force-calculation algorithm. *Nature*, *324*(6096), 446–449. https://doi.org/10.1038/324446a0

Batcher, K. E. (1968). Sorting Networks and Their Applications. *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, 307–314. https://doi.org/10.1145/1468075.1468121

Binney, J., & Tremaine, S. (2008). *Galactic Dynamics: Second Edition* (REV - Revised, 2). Princeton University Press. http://www.jstor.org/stable/j.ctvc778ff

Burtscher, M., & Pingali, K. (2011). An Efficient CUDA Implementation of the Tree-Based Barnes Hut n-Body Algorithm. *GPU Computing Gems Emerald Edition*, . https://doi.org/10.1016/B978-0-12-384988-5.00006-1

Congote, J., Segura, A., Kabongo, L., Moreno, A., Posada, J., & Ruiz, O. (2011). Interactive visualization of volumetric data with WebGL in real-time. *Proceedings of the 16th International Conference on 3d Web Technology*, 137–146. https://doi.org/10.1145/2010425.2010449

Emscripten contributors. (2024,). *Emscripten: An LLVM-to-WebAssembly compiler*. https://emscripten.org/

Fluke, C. J., Barnes, D. G., Barsdell, B. R., & Hassan, A. H. (2011). Astrophysical Supercomputing with GPUs: Critical Decisions for Early Adopters. *Publications of the Astronomical Society of Australia*, *28*(1), 15–27. https://doi.org/10.1071/as10019

Freeman, K. C. (1970). On the Disks of Spiral and S0 Galaxies. *The Astrophysical Journal*, *160*, 811. https://doi.org/10.1086/150474

Gaburov, E., Bédorf, J., & Portegies Zwart, S. (2010). Gravitational tree-code on graphics processing units: Implementation in CUDA. *Procedia Computer Science*, *324*, 1119–1127. https://doi.org/10.1016/j.procs.2010.04.124

gfx-rs contributors. (2024,). *wgpu: Safe and portable GPU abstraction in Rust*. https://github.com/gfx-rs/wgpu

Gomes, F. C. (2024,). *UniSim: Barnes–Hut N-body simulation in Metal*. https://github.com/fercgomes/unisim

Google. (2024,). *Dawn: Google's open-source WebGPU implementation*. https://dawn.googlesource.com/dawn

GPU for the Web Working Group. (2026,). *WebGPU (GPU for the Web) Specification*. https://gpuweb.github.io/gpuweb/

Karras, T. (2012). Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. *Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics*, 33–37.

Khronos Group. (2024b,). *Vulkan Specification*. https://www.vulkan.org/

Khronos Group. (2024a,). *WebGPU Specification*. https://www.w3.org/TR/webgpu/

Kreyszig, E. (2011). *Advanced Engineering Mathematics* (10th ed.). Wiley.

LF, G., & Rokhlin, V. (2001). A Fast Algorithm for Particle Simulation. *Journal of Computational Physics*, *73*, 325–348. https://doi.org/10.1016/0021-9991(87)90140-9

Maczan, J. (2026). *Characterizing WebGPU Dispatch Overhead for LLM Inference Across Four GPU Vendors, Three Backends, and Three Browsers*.

Morton, G. M. (1966). *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*.

Nyland, L., Harris, M., & Prins, J. (2009). Fast N-body simulation with CUDA. *GPU Gem, Vol. 3*, 677–695.

Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., & Purcell, T. J. (2007). A Survey of General-Purpose Computation on Graphics Hardware. *Computer Graphics Forum*, *26*(1), 80–113. https://doi.org/https://doi.org/10.1111/j.1467-8659.2007.01012.x

Owens, J., Houston, M., Luebke, D., Green, S., Stone, J., & Phillips, J. (2008). GPU computing. *Proceedings of the IEEE*, *96*, 879–899. https://doi.org/10.1109/JPROC.2008.917757

Peláez, R. P. (2022). *Complex Fluids in the GPU Era: Algorithms and Simulations*. https://raw.githubusercontent.com/RaulPPelaez/tesis/main/manuscript.pdf

Plummer, H. C. (1911). On the Problem of Distribution in Globular Star Clusters. *Monthly Notices of the Royal Astronomical Society*, *71*(5), 460–470. https://doi.org/10.1093/mnras/71.5.460

Salmon, J. K., & Warren, M. S. (1994). Skeletons from the Treecode Closet. *Journal of Computational Physics*, *111*(1), 136–155. https://doi.org/https://doi.org/10.1006/jcph.1994.1050

Sengupta, S., Wu, N., Varvello, M., Jana, K., Chen, S., & Han, B. (2025). From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration. *Proceedings of the 2025 ACM Internet Measurement Conference*, 1018–1024. https://doi.org/10.1145/3730567.3764504

Springel, V. (2005). The cosmological simulation code gadget-2. *Monthly Notices of the Royal Astronomical Society*, *364*(4), 1105–1134. https://doi.org/10.1111/j.1365-2966.2005.09655.x

Sung, N.-J., Ma, J., Kim, T., Choi, Y.-j., Choi, M.-H., & Hong, M. (2025,). *Real-Time Cloth Simulation Using WebGPU: Evaluating Limits of High-Resolution*. https://arxiv.org/abs/2507.11794

Sutter, H. (2005). The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. *Dr. Dobb's Journal*, *30*(3), 202–210. http://www.gotw.ca/publications/concurrency-ddj.htm

Usta, Z. (2024). WEBGPU: A NEW GRAPHIC API FOR 3D WEBGIS APPLICATIONS. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 377–382. https://doi.org/10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024

Verlet, L. (1967). Computer ``Experiments'' on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. *Physical Review*, *159*(1), 98–103. https://doi.org/10.1103/PhysRev.159.98

Wang, Q. (2021). A hybrid Fast Multipole Method for cosmological N-body simulations. *Research in Astronomy and Astrophysics*, *21*(1), 3. https://doi.org/10.1088/1674-4527/21/1/3

Zwart, S. F. P., Belleman, R. G., & Geldof, P. M. (2007). High-performance direct gravitational N-body simulations on graphics processing units. *New Astronomy*, *12*(8), 641–650. https://doi.org/https://doi.org/10.1016/j.newast.2007.05.004